

UNIVERSIDAD ABIERTA INTERAMERICANA

Facultad de Tecnología Informática

TurnoSync

Sistema de Gestión de Turnos Médicos

Carpeta de Documentación Técnica — Versión 2.0

Materia: Ingeniería de Software I

Institución: UAI — Sede Castelar

Alumno: Mendez, Federico Oscar

Legajo: B00019353

Docente: Rodriguez Escobedo Julian Martin

Localización / Comisión / Turno: Castelar / 3A / Noche

Año de cursada: 2026

Historial de Revisión

Fecha	Versión	Autor	Descripción
22/04/2026	1.0	Federico Mendez	Creación del documento. Entrega 1: T01, T02, T03.
01/05/2026	1.5	Federico Mendez	Entrega 2: T04 Composite, T05 Observer.
10/06/2026	2.0	Federico Mendez	Entrega 3: T06 Bitácora, T07 DVH/DVV. Versión final.

T01. Arquitectura Base

Objetivo

Definir la estructura técnica de TurnoSync mediante una arquitectura de 4 capas desacopladas, alineada con los principios SOLID y las buenas prácticas de orientación a objetos.

Stack Tecnológico

Componente	Tecnología / Herramienta
Lenguaje	C# (.NET Framework 4.7.2)
Interfaz de usuario	Windows Forms (MDI)
Base de datos	SQL Server LocalDB
Acceso a datos	ADO.NET (SqlConnection, SqlCommand, SqlDataReader)
IDE	Visual Studio 2022
Control de versiones	Git (repositorio local)

Descripción de las Capas

UI — Interfaz de Usuario

Formularios Windows Forms. Se comunica exclusivamente con BLL. El frmPrincipal actúa como contenedor MDI y verifica permisos antes de abrir formularios hijo.

BLL — Lógica de Negocio

Centraliza validaciones, cálculo de DVH/DVV, gestión de permisos, encriptado, bitácora e idioma. Único punto de comunicación entre UI y DAL.

DAL — Acceso a Datos

Gestiona la persistencia mediante ADO.NET. Expone métodos Insertar, Modificar, Eliminar, ObtenerTodos, ObtenerPorId por entidad.

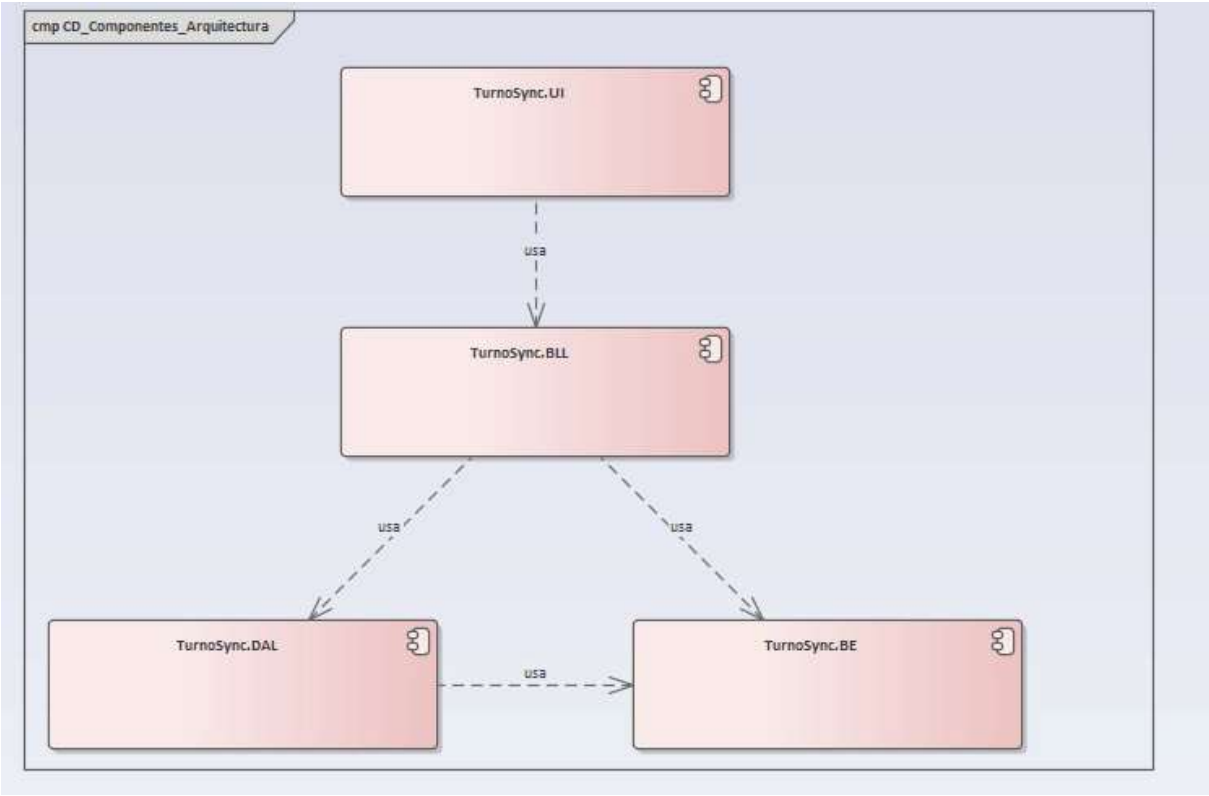
BE — Entidades de Negocio

Clases POCO que modelan el dominio. Usadas como objetos de transferencia entre todas las capas.

Diagrama de Componentes

La siguiente descripción representa la arquitectura de componentes del sistema:

Proyecto / Ensamblado	Responsabilidad
TurnoSync.UI	Formularios MDI, controles, navegación
TurnoSync.BLL	Lógica de negocio, validaciones, patrones
TurnoSync.DAL	Acceso a datos via ADO.NET
TurnoSync.BE	Entidades POCO de dominio



Mapa de Navegación

Código	Formulario	Descripción
UI-00	frmLogin	Autenticación con SHA-256
UI-01	frmPrincipal (MDI)	Contenedor MDI. Control de acceso por permisos
UI-02	frmPacientes	ABM de pacientes con DVH/DVV
UI-03	frmProfesionales	ABM de profesionales médicos
UI-04	frmEspecialidades	ABM de especialidades médicas
UI-05	frmTurnos	Gestión de turnos con validación de solapamiento
UI-06	frmUsuarios	ABM de usuarios del sistema
UI-07	frmRoles	Gestión del árbol Composite de permisos (TreeView)
UI-08	frmBitacora	Consulta y búsqueda de registros de bitácora

T02. Gestión de Log In / Log Out del Sistema

Objetivo

Verificar la identidad del usuario mediante nombre de usuario y contraseña encriptada, asignando los permisos correspondientes a su perfil.

Descripción del Proceso de Login

Al iniciar la aplicación, antes de mostrar frmLogin, el sistema ejecuta la verificación de integridad DVH/DVV. Si falla, se avisa al administrador y la aplicación se cierra.

- El usuario ingresa nombre de usuario y contraseña en frmLogin.
- La BLL delega el hash a clsSeguridadBLL que calcula SHA-256.
- La DAL compara el hash con el almacenado en la tabla Usuario.
- Si coincide, se instancia clsSesionActual (Singleton) con los permisos del usuario.
- Se registra el evento LOGIN en la tabla Bitacora.
- Se abre frmPrincipal que verifica permisos antes de habilitar ítems del menú.

Proceso de Logout / Cierre

Al cerrar la sesión se registra LOGOUT en bitácora, se limpia clsSesionActual y se vuelve al formulario de login.

Diagrama de Secuencia — Login

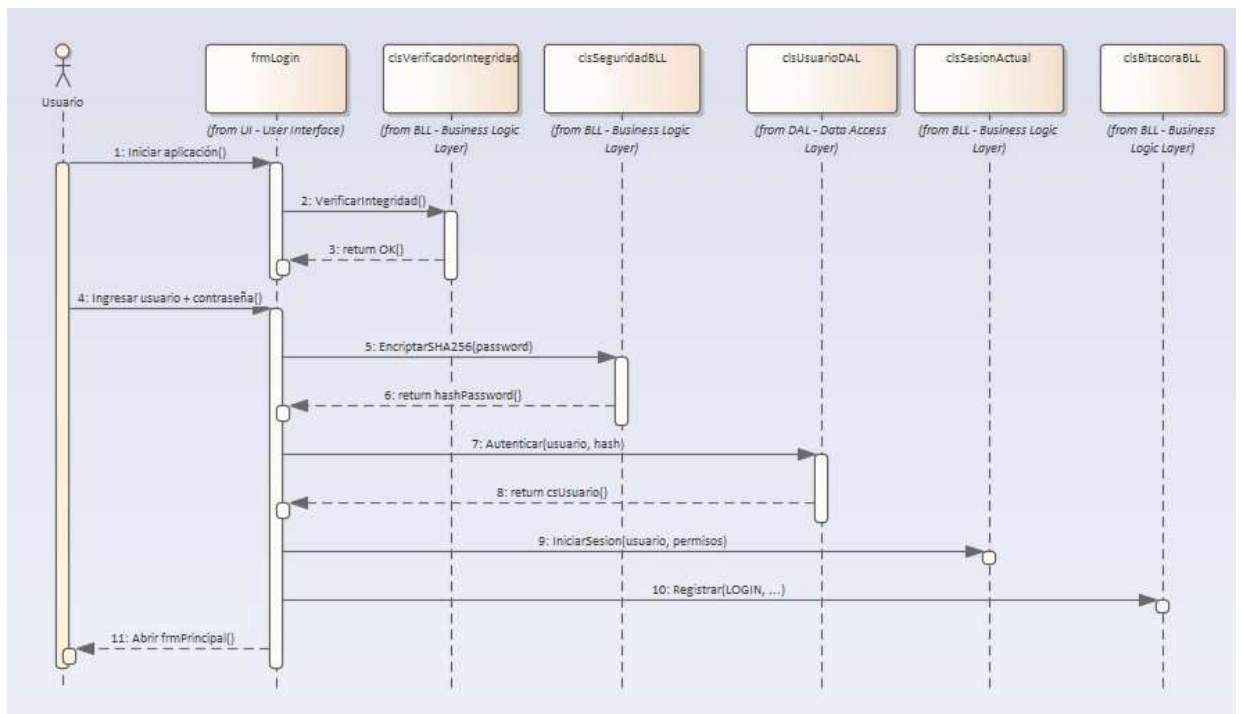
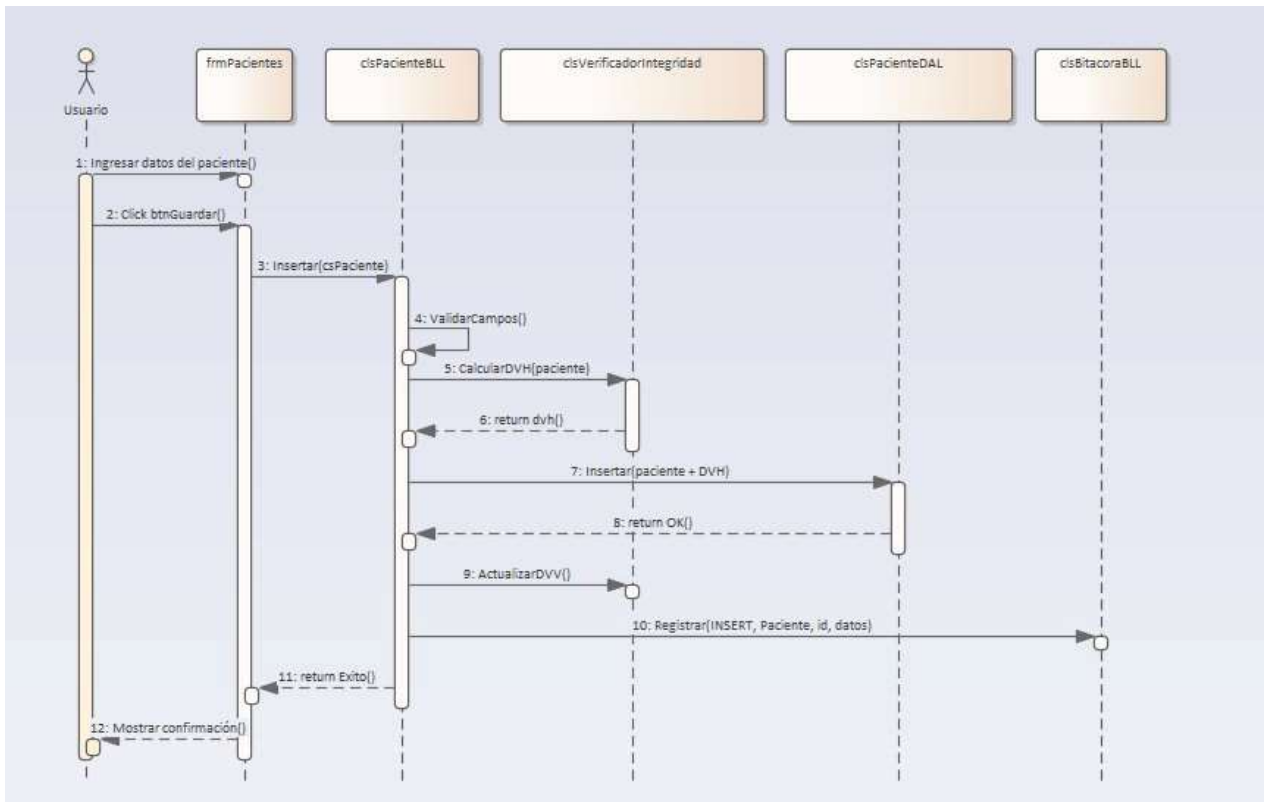


Diagrama de Secuencia — Alta Paciente



Patrón Singleton — clsSesionActual

Garantiza una única instancia de sesión por proceso. Almacena el usuario autenticado y sus permisos activos.

```

public class clsSesionActual {
    private static clsSesionActual _instancia;
    public csUsuario UsuarioActual { get; private set; }
    public List<string> PermisosActivos { get; private set; }
    private clsSesionActual() { }
    public static clsSesionActual Instancia {
        get { if (_instancia == null) _instancia = new clsSesionActual();
        return _instancia; }
    }
    public void IniciarSesion(csUsuario u, List<string> p) { UsuarioActual =
    u; PermisosActivos = p; }
    public void CerrarSesion() { UsuarioActual = null; PermisosActivos =
    null; _instancia = null; }
}
    
```

T03. Gestión de Encriptado

Objetivo

Proteger las contraseñas almacenadas en la base de datos mediante un algoritmo hash unidireccional.

Algoritmo: SHA-256

TurnoSync utiliza SHA-256 como mecanismo de hash. Es una función unidireccional: dado un hash, es computacionalmente inviable obtener la contraseña original.

Clase clsSeguridadBLL

```
public class clsSeguridadBLL {
    public string EncriptarSHA256(string texto) {
        using (SHA256 sha256 = SHA256.Create()) {
            byte[] bytes = Encoding.UTF8.GetBytes(texto);
            byte[] hash = sha256.ComputeHash(bytes);
            StringBuilder sb = new StringBuilder();
            foreach (byte b in hash) sb.Append(b.ToString("x2"));
            return sb.ToString();
        }
    }
}
```

Flujo de encriptado

- El usuario ingresa su contraseña en frmLogin.
- frmLogin llama a clsSeguridadBLL.EncriptarSHA256() antes de enviar a la DAL.
- La DAL compara el hash con el campo Contraseña de la tabla Usuario.
- Las contraseñas nunca viajan en texto plano entre capas.

T04. Gestión de Perfiles de Usuario — Patrón Composite

Objetivo

Implementar un sistema de permisos jerárquicos que permita definir permisos atómicos y compuestos, asignables dinámicamente a los usuarios.

Descripción del Patrón Composite

El patrón Composite permite tratar objetos individuales y composiciones de manera uniforme a través de una interfaz común. Un permiso puede ser atómico (funcionalidad individual) o compuesto (grupo de permisos).

Estructura de Clases

Clase / Interfaz	Tipo	Responsabilidad
IComponenteRol	Interfaz (Component)	Define: ObtenerNombre(), ObtenerCodigo(), EsHoja(), ObtenerHijos()
csRolSimple	Clase (Leaf)	Permiso atómico. Sin hijos. Implementa IComponenteRol.
csRolGrupo	Clase (Composite)	Permiso compuesto. Lista de hijos IComponenteRol. Implementa Agregar/Eliminar.
clsRolBLL	Clase BLL	Construye el árbol desde BD. Verifica permisos recursivamente.

Árbol de Permisos

- PA001 — Gestionar Pacientes
- PR001 — Gestionar Profesionales
- ES001 — Gestionar Especialidades
- TU001 — Gestionar Turnos
- US001 — Gestionar Usuarios
- RO001 — Gestionar Roles
- BIT001 — Consultar Bitácora

Perfiles compuestos:

- AA099 — Administrador: todos los permisos.
- MED001 — Médico: TU001, PA001, ES001.
- REC001 — Recepcionista: TU001, PA001.

Fragmento de Código

```
public interface IComponenteRol {
    string ObtenerNombre(); string ObtenerCodigo();
    bool EsHoja(); List<IComponenteRol> ObtenerHijos();
}
public class csRolSimple : IComponenteRol {
    public string Nombre { get; set; } public string Codigo { get; set; }
    public bool EsHoja() => true; public List<IComponenteRol> ObtenerHijos()
=> null;
}
public class csRolGrupo : IComponenteRol {
    private List<IComponenteRol> _hijos = new List<IComponenteRol>();
    public void Agregar(IComponenteRol r) => _hijos.Add(r);
    public bool EsHoja() => false;
    public List<IComponenteRol> ObtenerHijos() => _hijos;
}
public bool TienePermiso(IComponenteRol nodo, string cod) {
    if (nodo.ObtenerCodigo() == cod) return true;
    if (!nodo.EsHoja()) foreach (var h in nodo.ObtenerHijos()) if
(TienePermiso(h, cod)) return true;
    return false;
}
```

T05. Gestión de Múltiples Idiomas — Patrón Observer

Objetivo

Implementar el cambio dinámico de idioma (español/inglés) en todos los formularios, de forma que al cambiar el idioma activo, todos los formularios abiertos actualicen sus textos automáticamente.

Descripción del Patrón Observer

El patrón Observer define una dependencia uno-a-muchos. Cuando el Subject cambia de estado, notifica automáticamente a todos sus Observers. En TurnoSync, el Subject es `clsGestorIdioma` y los Observers son todos los formularios.

Estructura de Clases

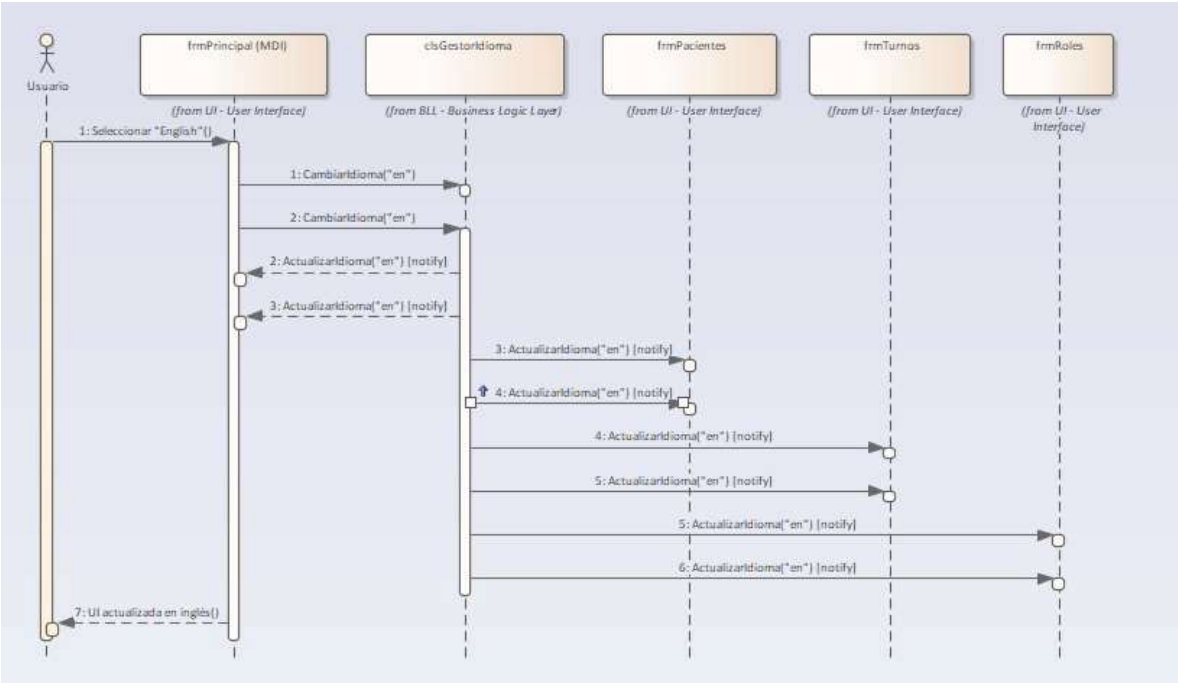
Clase / Interfaz	Rol	Responsabilidad
<code>IObservadorIdioma</code>	Observer (interfaz)	Define el contrato: <code>ActualizarIdioma(string idioma)</code>
<code>clsGestorIdioma</code>	Subject (Singleton)	Mantiene idioma activo. Gestiona observadores. Notifica cambios.
<code>frmPrincipal</code> , <code>frmPacientes</code> , etc.	Concrete Observer	Implementan <code>IObservadorIdioma</code> . Actualizan labels al recibir notificación.

Fragmento de Código

```
public interface IObservadorIdioma { void ActualizarIdioma(string idioma); }
public class clsGestorIdioma {
    private List<IObservadorIdioma> _obs = new List<IObservadorIdioma>();
    public void Registrar(IObservadorIdioma o) => _obs.Add(o);
    public void Desregistrar(IObservadorIdioma o) => _obs.Remove(o);
    public void CambiarIdioma(string idioma) { foreach (var o in _obs)
        o.ActualizarIdioma(idioma); }
}
```

Diagrama de Secuencia simplificado — Cambio de Idioma

1. El usuario selecciona "English" desde el menú en `frmPrincipal`.
2. `frmPrincipal` llama a `clsGestorIdioma.Instancia.CambiarIdioma("en")`.
3. `clsGestorIdioma` recorre su lista de observadores registrados.
4. Cada formulario abierto recibe `ActualizarIdioma("en")` y actualiza sus labels/botones.



T06. Gestión de Bitácora y Control de Cambios

T06a. Gestión de Bitácora

Objetivo

Registrar todas las operaciones del sistema generando un historial completo y auditable. Permite trazabilidad de acciones con búsqueda combinada por múltiples filtros.

Estructura de la tabla Bitacora

Campo	Tipo	Descripción
IdBitacora	INT (PK, IDENTITY)	Identificador autoincremental
Fecha	DATETIME	Fecha y hora de la operación
IdUsuario	INT (FK)	Usuario que realizó la acción
Operacion	VARCHAR(50)	INSERT, UPDATE, DELETE, LOGIN, LOGOUT
Entidad	VARCHAR(50)	Tabla afectada (ej: Paciente, Turno)
IdEntidad	INT	ID del registro afectado
ValorAntes	NVARCHAR(MAX)	Estado del objeto ANTES de la modificación
ValorDespues	NVARCHAR(MAX)	Estado del objeto DESPUÉS de la modificación

Implementación en BLL

```
public void Registrar(string operacion, string entidad, int id, string antes,
string despues) {
    csBitacora reg = new csBitacora {
        Fecha = DateTime.Now,
        IdUsuario = clsSesionActual.Instancia.UsuarioActual.IdUsuario,
        Operacion = operacion, Entidad = entidad, IdEntidad = id,
        ValorAntes = antes, ValorDespues = despues
    };
    new clsBitacoraDAL().Insertar(reg);
}
```

T06b. Control de Cambios

Objetivo

Registrar el estado ANTES y DESPUÉS de cada modificación, respondiendo a: ¿quién?, ¿cuándo? y ¿qué cambió?.

Estrategia ANTES/DESPUÉS

```
public void Modificar(csPaciente pNuevo) {
    csPaciente anterior = new
clsPacienteDAL().ObtenerPorId(pNuevo.IdPaciente);
    string antes = SerializarPaciente(anterior);
    new clsPacienteDAL().Modificar(pNuevo);
    string despues = SerializarPaciente(pNuevo);
    new clsBitacoraBLL().Registrar("UPDATE", "Paciente", pNuevo.IdPaciente,
antes, despues);
}
```

T07. Gestión de Dígitos Verificadores (DVH y DVV)

Objetivo

Garantizar la integridad de los datos detectando: registros modificados por fuera del sistema y transposición de caracteres en campos.

Conceptos

Término	Definición
DVH (Horizontal)	Dígito calculado por registro. Se almacena en la columna DVH de la entidad.
DVV (Vertical)	Dígito calculado para todos los DVH de una tabla. Se almacena en DigitoVerificador.

Algoritmo — Módulo 97 con Pesos Posicionales

- Por cada campo: multiplicar ASCII × posición en campo × posición del campo.
- Acumular todos los productos.
- $DVH = suma_total \% 97$

Ejemplo Numérico

Paciente: Nombre = "ANA" (campo 1), DNI = "123" (campo 2)

$'A' (65) \times 1 \times 1 = 65$ | $'N' (78) \times 2 \times 1 = 156$ | $'A' (65) \times 3 \times 1 = 195 \rightarrow 416$
 $'1' (49) \times 1 \times 2 = 98$ | $'2' (50) \times 2 \times 2 = 200$ | $'3' (51) \times 3 \times 2 = 306 \rightarrow 604$
 $TOTAL = 1020 \rightarrow DVH = 1020 \% 97 = 50$

DVV con 3 registros DVH [50, 35, 72]:

$DVV = (50 \times 1 + 35 \times 2 + 72 \times 3) \% 97 = 336 \% 97 = 45$

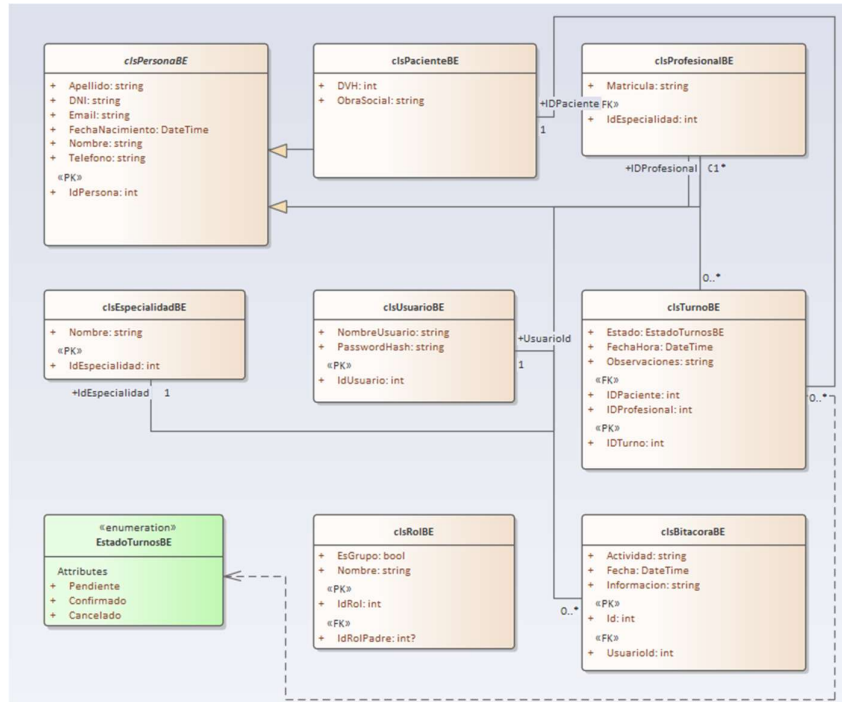
Fragmento de Código — Cálculo DVH

```
public int CalcularDVH(csPaciente p) {
    string[] campos = { p.Nombre, p.Apellido, p.DNI, p.Telefono, p.Email };
    int suma = 0;
    for (int i = 0; i < campos.Length; i++) {
        string val = campos[i] ?? "";
        for (int j = 0; j < val.Length; j++)
            suma += (int)val[j] * (j+1) * (i+1);
    }
    return suma % 97;
}
```

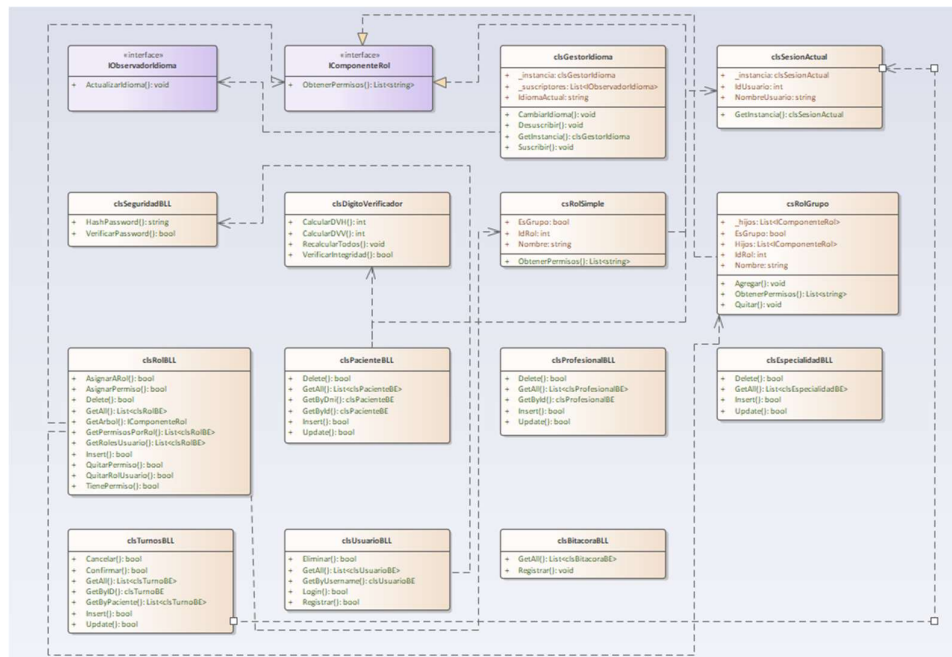
G07. Diagramas de Clases por Capa

Los siguientes diagramas corresponden a las clases de cada capa de la arquitectura de TurnoSync.

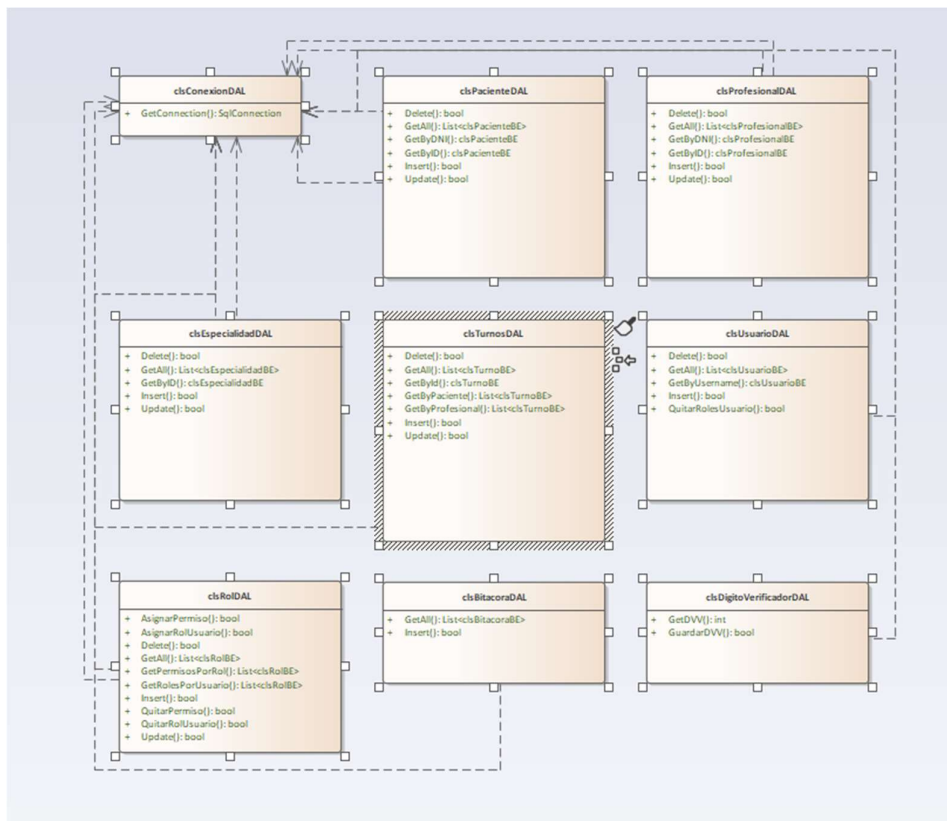
Capa BE — Entidades de Negocio



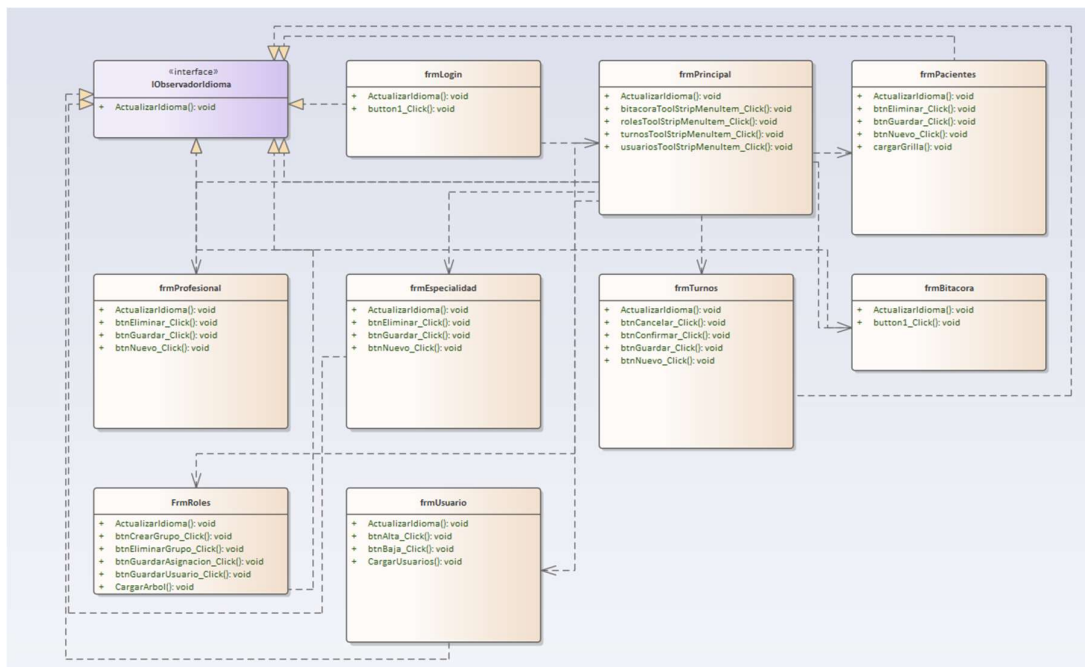
Capa BLL — Lógica de Negocio (Patrones)



Capa DAL — Acceso a Datos



Capa UI — Interfaz de Usuario (Observer)



G08. Modelo de Datos — Diagrama Entidad-Relació

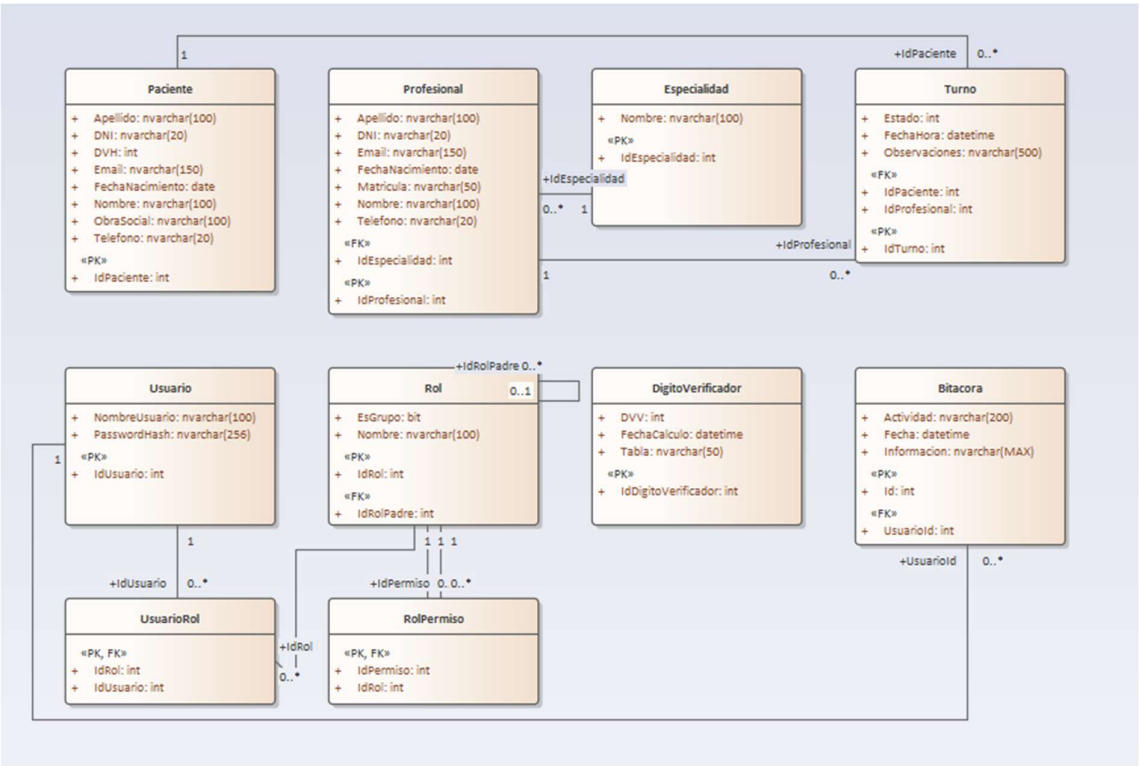
Descripción de Tablas

Tabla	Campos Principales
Paciente	IdPaciente (PK), Nombre, Apellido, DNI, FechaNacimiento, Telefono, Email, DVH
Profesional	IdProfesional (PK), Nombre, Apellido, Matricula, IdEspecialidad (FK), Telefono, Email
Especialidad	IdEspecialidad (PK), Descripcion
Turno	IdTurno (PK), IdPaciente (FK), IdProfesional (FK), Fecha, HoraInicio, HoraFin, Estado
Usuario	IdUsuario (PK), NombreUsuario, Contraseña (hash SHA-256), Activo
Rol	IdRol (PK),Codigo, Nombre, EsGrupo, IdRolPadre (FK self-referential)
UsuarioRol	IdUsuario (FK), IdRol (FK) — tabla unión N:M
Bitacora	IdBitacora (PK), Fecha, IdUsuario (FK), Operacion, Entidad, IdEntidad, ValorAntes, ValorDespues
DigitoVerificador	IdDigitoVerificador (PK), NombreTabla, ValorDVV, FechaCalculo

Relaciones Clave

- Turno → Paciente (N:1): Un paciente puede tener múltiples turnos.
- Turno → Profesional (N:1): Un profesional puede tener múltiples turnos.
- Profesional → Especialidad (N:1): Un profesional pertenece a una especialidad.
- Usuario ↔ Rol (N:M vía UsuarioRol).
- Rol → Rol (auto-referencial): jerarquía Composite.
- Bitacora → Usuario (N:1).

Diagrama Entidad-Relación



N02. Especificaciones de Casos de Uso

CU-01: Iniciar Sesión

Campo	Descripción
ID	CU-01
Nombre	Iniciar Sesión
Actor	Usuario del sistema
Precondición	La aplicación está iniciada. Verificación DVH/DVV exitosa.
Flujo Principal	1. El sistema muestra frmLogin. 2. El usuario ingresa usuario y contraseña. 3. El sistema encripta con SHA-256. 4. Verifica credenciales en BD. 5. Carga permisos. 6. Inicializa clsSesionActual. 7. Registra LOGIN en Bitácora. 8. Abre frmPrincipal.
Flujo Alternativo	4a. Credenciales incorrectas: mensaje de error. 3 intentos fallidos bloquean el acceso.
Postcondición	Usuario autenticado con sesión y permisos activos.

CU-02: Gestionar Pacientes

Campo	Descripción
ID	CU-02
Nombre	Gestionar Pacientes
Actor	Recepcionista / Administrador
Precondición	Usuario autenticado con permiso PA001.
Flujo Principal	1. Abre frmPacientes. 2. Ingresar datos. 3. Valida campos obligatorios. 4. Calcula DVH. 5. Inserta con DVH. 6. Recalcula DVV. 7. Actualiza DigitoVerificador. 8. Registra en Bitácora.
Flujo Alternativo	3a. Datos inválidos o DNI duplicado: informa el error.
Postcondición	Paciente registrado con DVH válido y DVV actualizado.

CU-03: Gestionar Profesionales

Campo	Descripción
ID	CU-03
Nombre	Gestionar Profesionales
Actor	Administrador
Precondición	Usuario autenticado con permiso PR001.
Flujo Principal	1. Abre frmProfesionales. 2. Ingresar datos del profesional. 3. Selecciona especialidad. 4. Valida matrícula única. 5. Inserta registro. 6. Registra en Bitácora.
Flujo Alternativo	4a. Matrícula duplicada: informa el error.
Postcondición	Profesional registrado en el sistema.

CU-04: Gestionar Especialidades

Campo	Descripción
ID	CU-04
Nombre	Gestionar Especialidades
Actor	Administrador
Precondición	Usuario autenticado con permiso ES001.
Flujo Principal	1. Abre frmEspecialidades. 2. Ingresa descripción. 3. Valida unicidad. 4. Inserta registro. 5. Registra en Bitácora.
Flujo Alternativo	3a. Especialidad duplicada: informa el error.
Postcondición	Especialidad registrada y disponible para asignar a profesionales.

CU-05: Gestionar Turnos

Campo	Descripción
ID	CU-05
Nombre	Gestionar Turnos
Actor	Recepcionista / Administrador
Precondición	Usuario autenticado con permiso TU001. Paciente y profesional registrados.
Flujo Principal	1. Abre frmTurnos. 2. Selecciona paciente, profesional, fecha y hora. 3. Verifica solapamiento de horarios. 4. Inserta turno con estado Pendiente. 5. Registra en Bitácora.
Flujo Alternativo	3a. Horario no disponible: mensaje de conflicto.
Postcondición	Turno registrado en la agenda del profesional.

CU-06: Gestionar Usuarios

Campo	Descripción
ID	CU-06
Nombre	Gestionar Usuarios
Actor	Administrador
Precondición	Usuario autenticado con permiso US001.
Flujo Principal	1. Abre frmUsuarios. 2. Ingresa datos del usuario. 3. La BLL encripta la contraseña con SHA-256. 4. Asigna rol/perfil vía UsuarioRol. 5. Inserta registro. 6. Registra en Bitácora.
Flujo Alternativo	3a. Nombre de usuario duplicado: informa el error.
Postcondición	Usuario registrado con contraseña encriptada y perfil asignado.

CU-07: Gestionar Roles y Permisos

Campo	Descripción
ID	CU-07
Nombre	Gestionar Roles y Permisos
Actor	Administrador
Precondición	Usuario autenticado con permiso RO001.
Flujo Principal	1. Abre frmRoles. 2. El sistema muestra el árbol Composite en TreeView. 3. El administrador crea/modifica roles simples o grupos. 4. Asigna permisos atómicos. 5. Guarda cambios en BD. 6. Registra en Bitácora.
Flujo Alternativo	Rol con nombre duplicado: informa el error.
Postcondición	Árbol de permisos actualizado.

CU-08: Consultar Bitácora

Campo	Descripción
ID	CU-08
Nombre	Consultar Bitácora
Actor	Administrador
Precondición	Usuario autenticado con permiso BIT001.
Flujo Principal	1. Abre frmBitacora. 2. Ingresar filtros combinados (fecha, usuario, operación, entidad). 3. El sistema ejecuta la búsqueda. 4. Muestra los registros encontrados con ValorAntes y ValorDespues.
Flujo Alternativo	Sin resultados: mensaje informativo.
Postcondición	Registros de bitácora filtrados y visibles para auditoría.

CU-09: Cambiar Idioma

Campo	Descripción
ID	CU-09
Nombre	Cambiar Idioma
Actor	Cualquier usuario autenticado
Precondición	Usuario autenticado. Al menos un formulario abierto.
Flujo Principal	1. El usuario selecciona el idioma desde el menú de frmPrincipal. 2. frmPrincipal llama a clsGestorIdioma.CambiarIdioma(). 3. clsGestorIdioma notifica a todos los observers registrados. 4. Cada formulario actualiza sus labels y botones.
Flujo Alternativo	No aplica.
Postcondición	Todos los formularios abiertos muestran el idioma seleccionado.

CU-10: Verificar Integridad DVH/DVV

Campo	Descripción
ID	CU-10
Nombre	Verificar Integridad DVH/DVV
Actor	Sistema (automático al iniciar)
Precondición	La aplicación se está iniciando.
Flujo Principal	1. clsVerificadorIntegridad recalcula el DVH de cada registro de Paciente. 2. Compara con el DVH almacenado. 3. Recalcula el DVV de la tabla. 4. Compara con el valor en DigitoVerificador. 5. Si todo OK: muestra frmLogin.
Flujo Alternativo	Error de integridad: avisa al administrador y cierra la aplicación.
Postcondición	Integridad verificada. Sistema listo para autenticación.

Apéndice A — Glosario y Definiciones

Término / Acrónimo	Definición
ABM	Alta, Baja y Modificación. Operaciones CRUD sobre una entidad.
BLL	Business Logic Layer. Capa de lógica de negocio.
DAL	Data Access Layer. Capa de acceso a datos.
BE	Business Entities. Capa de entidades/modelos de dominio.
DVH	Dígito Verificador Horizontal. Valor por registro.
DVV	Dígito Verificador Vertical. Valor por tabla.
MDI	Multiple Document Interface. Ventana padre con múltiples ventanas hijo.
SHA-256	Secure Hash Algorithm 256 bits. Función hash criptográfica unidireccional.
Singleton	Patrón creacional que garantiza una única instancia por proceso.
Composite	Patrón estructural para jerarquías parte-todo.
Observer	Patrón de comportamiento para dependencias uno-a-muchos.
TurnoSync	Sistema de Gestión de Turnos Médicos desarrollado en C# WinForms.